

JavaScript Intermediate — Advanced Class Features & Patterns

Section 15.4 — `static`, Private Fields, `instanceof`, Property Ownership & Mixins

এই section-এ আমরা শিখব:

1. `static` methods & properties
2. Private class fields (`#`)
3. `instanceof`
4. `hasOwnProperty()` vs `Object.hasOwnProperty()`
5. Mixins pattern with classes

এগুলোর সাথে related গুরুত্বপূর্ণ concepts-ও থাকবে।

1. `static` কী?

Class-এর সাধারণ methods instance-এর মাধ্যমে ব্যবহার করা হয়:

```
class User {  
  greet() {  
    return "Hello";  
  }  
}  
  
const user = new User();  
  
console.log(user.greet());
```

কিন্তু `static` method instance-এর জন্য নয়।

এটা `class`-এর জন্য।

```
class User {  
  static createGuest() {  
    return new User("Guest");  
  }  
}
```

```
    constructor(name) {  
      this.name = name;  
    }  
  }  
  
  const user = User.createGuest();  
  
  console.log(user.name);
```

Output:

Guest

এখানে:

User.createGuest()

সঠিক।

কিন্তু:

user.createGuest()

ভুল।

2. Instance Method vs Static Method

Instance method

```
class User {  
  greet() {  
    return `Hello ${this.name}`;  
  }  
}
```

ব্যবহার:

```
const user = new User();

user.greet();
```

Static method

```
class User {
  static createGuest() {
    return new User();
  }
}
```

ব্যবহার:

```
User.createGuest();
```

Mental Model

```
Instance method
  ↓
instance.method()

Static method
  ↓
Class.method()
```

3. Static Method কোথায় থাকে?

ধরো:

```
class User {
  greet() {
    return "Hello";
  }

  static createGuest() {
    return new User("Guest");
  }
}
```

```
}  
}
```

Conceptually:

```
User  
|  
├── static createGuest()  
└── prototype  
    │  
    └── greet()
```

অর্থাৎ:

```
User.createGuest()
```

এবং:

```
user.greet()
```

দুইটি আলাদা জায়গা থেকে আসে।

4. Static Method-এর Practical Use

Static method useful যখন কোনো behavior specific instance-এর data-এর ওপর depend করে না।

Example:

```
class MathHelper {  
  static add(a, b) {  
    return a + b;  
  }  
  
  static multiply(a, b) {  
    return a * b;  
  }  
}
```

```
console.log(MathHelper.add(10, 20));  
console.log(MathHelper.multiply(5, 4));
```

Output:

```
30  
20
```

এখানে `MathHelper` instance বানানোর দরকার নেই।

5. Static Factory Method

Static methods-এর একটি খুব common use হলো **factory method**।

```
class User {  
  constructor(name, role) {  
    this.name = name;  
    this.role = role;  
  }  
  
  static createAdmin(name) {  
    return new User(name, "admin");  
  }  
  
  static createGuest(name) {  
    return new User(name, "guest");  
  }  
}
```

Use:

```
const admin = User.createAdmin("Duke");  
const guest = User.createGuest("Rahim");  
  
console.log(admin);  
console.log(guest);
```

এখানে static methods object creation simplify করছে।

6. Static Properties

Static শুধু method-এর জন্য নয়।

Class-এর static property-ও থাকতে পারে।

```
class User {  
  static role = "user";  
}  
  
console.log(User.role);
```

Output:

```
user
```

কিন্তু:

```
const user = new User();  
  
console.log(user.role);
```

Output:

```
undefined
```

কারণ `role` static property।

7. Instance Property vs Static Property

```
class User {  
  static type = "USER";  
  
  constructor(name) {  
    this.name = name;  
  }  
}
```

এখানে:

```
User.type
```

class-level property।

আর:

```
user.name
```

instance-level property।

Mental model:

```
User
└─ static type

user
└─ name
```

8. Static Initialization Blocks

Modern JavaScript-এ static initialization block-ও আছে।

```
class Config {
  static version;

  static {
    Config.version = "1.0.0";
  }
}

console.log(Config.version);
```

Static block class definition-এর সময় execute হয়।

এটা advanced use case।

সাধারণ application code-এ static method/property অনেক বেশি common ।

9. Private Class Fields —

JavaScript class-এ সত্যিকারের private field তৈরি করতে # ব্যবহার করা হয় ।

```
class BankAccount {  
  #balance = 0;  
  
  deposit(amount) {  
    this.#balance += amount;  
  }  
  
  getBalance() {  
    return this.#balance;  
  }  
}
```

Use:

```
const account = new BankAccount();  
  
account.deposit(1000);  
  
console.log(account.getBalance());
```

Output:

```
1000
```

10. Private Field বাইরে থেকে Access করা যায় না

এটা:

```
account.#balance
```

✗ Syntax error ।

কারণ `#balance` সত্যিকারের private ।

শুধু class-এর ভিতর থেকে access করা যায় ।

```
class BankAccount {  
  #balance = 0;  
  
  deposit(amount) {  
    this.#balance += amount;  
  }  
}
```

11. `#` বনাম `_`

অনেক পুরনো code-এ দেখা যায়:

```
this._balance
```

`_` শুধু convention ।

এটা technically private নয় ।

```
const account = new BankAccount();  
  
console.log(account._balance);
```

access করা সম্ভব হতে পারে ।

কিন্তু:

```
#balance
```

সত্যিকারের private field ।

Difference

Syntax	Meaning
<code>_balance</code>	convention অনুযায়ী internal

Syntax	Meaning
#balance	language-level private

12. Private Method

শুধু field নয়, private method-ও তৈরি করা যায়।

```
class User {
  #validateName(name) {
    return name.length > 2;
  }

  setName(name) {
    if (this.#validateName(name)) {
      this.name = name;
    }
  }
}
```

এখানে:

```
#validateName()
```

শুধু class-এর ভিতরে ব্যবহারযোগ্য।

13. Private Getter/Setter

Private accessor-ও possible।

```
class User {
  #name = "Duke";

  get #userName() {
    return this.#name;
  }

  printName() {
    console.log(this.#userName);
  }
}
```

```
}  
}
```

এটা advanced feature ।

সাধারণ ক্ষেত্রে private field + public getter/setter বেশি readable ।

14. Private Static Fields

Static field-ও private হতে পারে ।

```
class Counter {  
  static #count = 0;  
  
  static increment() {  
    Counter.#count++;  
  }  
  
  static getCount() {  
    return Counter.#count;  
  }  
}
```

Use:

```
Counter.increment();  
Counter.increment();  
  
console.log(Counter.getCount());
```

Output:

2

15. Private Fields এবং Encapsulation

Private fields-এর মূল উদ্দেশ্য:

Internal state বাইরে থেকে **direct modification** থেকে **protect** করা ।

Example:

```
class BankAccount {  
  #balance = 0;  
  
  deposit(amount) {  
    if (amount <= 0) {  
      throw new Error("Invalid amount");  
    }  
  
    this.#balance += amount;  
  }  
  
  getBalance() {  
    return this.#balance;  
  }  
}
```

এখন বাইরে থেকে কেউ সরাসরি:

```
account.#balance = -100000;
```

করতে পারবে না।

এটা **encapsulation**-এর basic example।

16. Private Field-এর Important Behavior

Private fields prototype-এ থাকে না।

এগুলো instance-এর internal private state।

```
class User {  
  #password = "secret";  
}
```

এটা:

```
Object.keys(user)
```

দিয়ে দেখা যাবে না।

এছাড়া:

```
"#password" in user
```

দিয়েও private field check করা যায় না।

Private names-এর জন্য আলাদা language-level semantics আছে।

17. instanceof কী?

`instanceof` দিয়ে check করা যায় কোনো object একটি constructor/class-এর prototype chain-এর instance কি না।

Syntax:

```
object instanceof Constructor
```

Example:

```
class User {}  
  
const user = new User();  
  
console.log(user instanceof User);
```

Output:

```
true
```

18. instanceof with Inheritance

```
class User {}  
  
class Admin extends User {}  
  
const admin = new Admin();
```

```
console.log(admin instanceof Admin);  
console.log(admin instanceof User);
```

Output:

```
true  
true
```

কারণ:

```
admin  
↓  
Admin.prototype  
↓  
User.prototype
```

19. instanceof Object

প্রায় সব normal object-এর ক্ষেত্রে:

```
const user = {};  
  
console.log(user instanceof Object);
```

Output:

```
true
```

Array-এর ক্ষেত্রেও:

```
const numbers = [];  
  
console.log(numbers instanceof Array);  
console.log(numbers instanceof Object);
```

Output:

```
true  
true
```

কারণ array-এর prototype chain শেষ পর্যন্ত `Object.prototype` -এ পৌঁছায়।

20. `instanceof` Prototype Chain-এর ওপর কাজ করে

ধরো:

```
class Animal {}  
  
class Dog extends Animal {}  
  
const dog = new Dog();
```

Prototype chain:

```
dog  
↓  
Dog.prototype  
↓  
Animal.prototype  
↓  
Object.prototype  
↓  
null
```

তাই:

```
dog instanceof Dog
```

→ `true`

```
dog instanceof Animal
```

→ `true`

```
dog instanceof Object
```

→ `true`

21. instanceof vs typeof

দুটোর purpose আলাদা।

`typeof`

Basic JavaScript type সম্পর্কে information দেয়।

```
typeof "hello"; // "string"
typeof 10;      // "number"
typeof true;    // "boolean"
```

`instanceof`

Object কোন constructor/class-এর prototype chain-এর সাথে connected কিনা check করে।

```
const user = new User();

user instanceof User;
```

22. typeof vs instanceof Table

Check	Example	Result
Primitive type	<code>typeof "hello"</code>	<code>"string"</code>
Number	<code>typeof 10</code>	<code>"number"</code>
Array	<code>typeof []</code>	<code>"object"</code>
Class instance	<code>user instanceof User</code>	<code>true</code>
Array	<code>arr instanceof Array</code>	<code>true</code>
Object	<code>obj instanceof Object</code>	<code>true</code>

Array check-এর জন্য সাধারণত:


```
Array.isArray(value)
```

সবচেয়ে clear choice ।

23. `hasOwnProperty()` কী?

আগের prototype section-এ আমরা দেখেছি own property এবং inherited property ।

Example:

```
const user = {  
  name: "Duke"  
};
```

Check:

```
console.log(user.hasOwnProperty("name"));
```

Output:

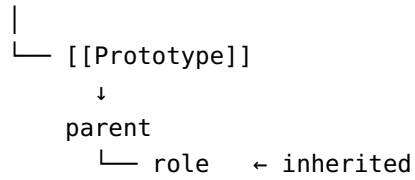
```
true
```

24. Own Property vs Inherited Property

```
const parent = {  
  role: "admin"  
};  
  
const child = Object.create(parent);  
  
child.name = "Duke";
```

এখন:

```
child  
├─ name      ← own
```



তাই:

```
console.log(child.hasOwnProperty("name"));  
console.log(child.hasOwnProperty("role"));
```

Output:

```
true  
false
```

25. Object.hasOwnProperty()

Modern JavaScript-এ own property check করার জন্য:

```
Object.hasOwnProperty(object, property)
```

ব্যবহার করা যায়।

Example:

```
const user = {  
  name: "Duke"  
};  
  
console.log(Object.hasOwnProperty(user, "name"));
```

Output:

```
true
```

26. `hasOwnProperty()` vs `Object.hasOwnProperty()`

Recommended modern approach:

```
Object.hasOwnProperty(obj, key);
```

কারণ এটা object-এর নিজের `hasOwnProperty` method-এর ওপর নির্ভর করে না।

Example:

```
const obj = Object.create(null);  
  
obj.name = "Duke";
```

এখানে:

```
obj.hasOwnProperty("name");
```

❌ কাজ করবে না।

কারণ `Object.create(null)` object-এর prototype নেই।

কিন্তু:

```
Object.hasOwnProperty(obj, "name");
```

কাজ করবে।

27. Object-এর `hasOwnProperty` Shadow হতে পারে

ধরো:

```
const user = {  
  name: "Duke",  
  hasOwnProperty: "hello"  
};
```

এখন:

```
user.hasOwnProperty("name");
```

কাজ করবে না, কারণ নিজের property parent-এর method-কে shadow করেছে।

কিন্তু:

```
Object.hasOwn(user, "name");
```

ঠিকভাবে কাজ করবে।

এটাই `Object.hasOwn()` ব্যবহারের একটি গুরুত্বপূর্ণ কারণ।

28. `in` vs `Object.hasOwn()`

এই difference খুব important।

```
const parent = {  
  role: "admin"  
};  
  
const user = Object.create(parent);  
  
user.name = "Duke";
```

`in`

```
console.log("name" in user);  
console.log("role" in user);
```

Output:

```
true  
true
```

`in` own + inherited দুইটাই check করে।

`Object.hasOwn()`

```
console.log(Object.hasOwn(user, "name"));  
console.log(Object.hasOwn(user, "role"));
```

Output:

```
true  
false
```

শুধু own property check করে।

29. Quick Comparison

Method	Own	Inherited
<code>Object.hasOwn(obj, key)</code>	✓	✗
<code>obj.hasOwnProperty(key)</code>	✓	✗
<code>key in obj</code>	✓	✓

Modern code-এ own-property check:

```
Object.hasOwn(obj, key)
```

খুব ভালো default choice।

30. Mixins কী?

JavaScript class inheritance সাধারণত single-parent model follow করে।

একটি class:

```
class Admin extends User {}
```

একটি parent class extend করে।

কিন্তু কখনও আমরা চাই একটি class বিভিন্ন reusable behavior পাবে।

যেমন:

```
User
├─ canLogin
├─ canLogout
├─ canTrack
└─ canNotify
```

এক্ষেত্রে **Mixins** useful হতে পারে।

31. Mixin-এর Basic Idea

Mixin হলো reusable behavior-এর একটি object/function যেটা অন্য class-এ behavior যোগ করে।

Example:

```
const canLogin = {
  login() {
    return "Logged in";
  }
};
```

আরেকটি:

```
const canLogout = {
  logout() {
    return "Logged out";
  }
};
```

এখন class:

```
class User {
  constructor(name) {
    this.name = name;
  }
}
```

Mixin apply:

```
Object.assign(User.prototype, canLogin, canLogout);
```

এখন:

```
const user = new User("Duke");  
  
console.log(user.login());  
console.log(user.logout());
```

Output:

```
Logged in  
Logged out
```

32. কেন Mixins দরকার?

ধরো:

```
User  
Admin  
Moderator  
Editor
```

সবাই login করতে পারে।

আবার:

```
Admin  
Moderator
```

notification পাঠাতে পারে।

Inheritance দিয়ে সব behavior model করা সবসময় clean হয় না।

Mixins দিয়ে:

```
Login behavior  
Notification behavior
```

Logging behavior
Permission behavior

reusable করা যায়।

33. Mixin Factory Pattern

আরও clean pattern:

```
const canLogin = (Base) =>
  class extends Base {
    login() {
      return `${this.name} logged in`;
    }
  };
```

আরেকটি:

```
const canLogout = (Base) =>
  class extends Base {
    logout() {
      return `${this.name} logged out`;
    }
  };
```

Base class:

```
class User {
  constructor(name) {
    this.name = name;
  }
}
```

Combine:

```
class Admin extends canLogout(canLogin(User)) {}
```

Use:


```
const admin = new Admin("Duke");

console.log(admin.login());
console.log(admin.logout());
```

34. Mixin Composition Mental Model

```
User
↓
canLogin(User)
↓
canLogout(...)
↓
Admin
```

অর্থাৎ behavior layer-by-layer compose হচ্ছে।

এটা multiple inheritance-এর মতো flexibility দিতে পারে, যদিও JavaScript-এর native class inheritance single superclass-based।

35. Mixin Example — Timestamp

ধরো আমরা যেকোনো object-এ timestamp behavior দিতে চাই।

```
const TimestampMixin = (Base) =>
  class extends Base {
    createdAt = new Date();

    getCreatedAt() {
      return this.createdAt;
    }
  };
```

Base:

```
class User {
  constructor(name) {
    this.name = name;
  }
}
```

```
}  
}
```

Use:

```
class TimestampedUser extends TimestampMixin(User) {}  
  
const user = new TimestampedUser("Duke");  
  
console.log(user.getCreatedAt());
```

এভাবে reusable behavior তৈরি করা যায়।

36. Mixin-এর সতর্কতা

Mixins powerful হলেও overuse করলে code difficult হতে পারে।

সমস্যা হতে পারে:

- একই method name collision
- behavior কোথা থেকে এসেছে বোঝা কঠিন
- inheritance chain complex হওয়া
- debugging difficult হওয়া
- hidden dependencies

তাই simple inheritance যথেষ্ট হলে mixin ব্যবহার করার দরকার নেই।

37. Static + Private Field Example

এখন কয়েকটি advanced feature একসাথে দেখি।

```
class User {  
  static #count = 0;  
  
  #id;  
  
  constructor(name) {  
    this.name = name;  
    this.#id = ++User.#count;  
  }  
}
```

```
getId() {  
    return this.#id;  
}  
  
static getUserCount() {  
    return User.#count;  
}  
}
```

Use:

```
const user1 = new User("Duke");  
const user2 = new User("Rahim");  
  
console.log(user1.getId());  
console.log(user2.getId());  
  
console.log(User.getUserCount());
```

Output:

```
1  
2  
2
```

এখানে:

```
User.#count
```

→ private static state ।

আর:

```
user.#id
```

→ private instance state ।

38. instanceof + Private Class

Private field check করার সময় একটি interesting behavior আছে ।

```
class User {  
  #id;  
  
  constructor() {  
    this.#id = 1;  
  }  
  
  hasPrivateId() {  
    return #id in this;  
  }  
}
```

এখানে:

```
#id in this
```

private field existence check করার language feature ।

এটা normal:

```
"id" in object
```

এর মতো নয় ।

39. Static Inheritance

Static methods child class-এও inherited access pattern পেতে পারে ।

```
class User {  
  static role() {  
    return "user";  
  }  
}  
  
class Admin extends User {}
```

এখন:

```
console.log(Admin.role());
```

Output:

```
user
```

কারণ class constructor objects-এর মধ্যেও inheritance relationship থাকে।

এটা instance prototype chain-এর পাশাপাশি class-level inheritance relationship বোঝার জন্য useful।

40. Static Method + `this`

Static method-এর ভিতরে `this` সাধারণত class/constructor-কে refer করে।

```
class User {  
  static create() {  
    return new this();  
  }  
}
```

এখন:

```
class Admin extends User {}  
  
const admin = Admin.create();  
  
console.log(admin instanceof Admin);
```

Output:

```
true
```

কারণ inherited static method-এ `this` subclass constructor-কে refer করতে পারে।

এটা factory pattern-এর advanced technique।

41. Object.hasOwn() Practical API Example

API data process করার সময়:

```
function hasField(data, field) {  
  return Object.hasOwn(data, field);  
}  
  
const user = {  
  name: "Duke",  
  age: 31  
};  
  
console.log(hasField(user, "name"));  
console.log(hasField(user, "email"));
```

Output:

```
true  
false
```

এতে inherited properties accidentally process করার সম্ভাবনা কমে।

42. Real-World Class Design

একটি practical example:

```
class Product {  
  static category = "General";  
  
  #price;  
  
  constructor(name, price) {  
    this.name = name;  
    this.#price = price;  
  }  
  
  get price() {  
    return this.#price;  
  }  
}
```

```

set price(value) {
  if (value < 0) {
    throw new Error("Price cannot be negative");
  }

  this.#price = value;
}

getInfo() {
  return `${this.name}: ${this.#price}`;
}

static createFreeProduct(name) {
  return new Product(name, 0);
}
}

```

Use:

```

const product = new Product("Keyboard", 50);

console.log(product.getInfo());
console.log(product.price);

product.price = 60;

console.log(product.price);

const freeProduct = Product.createFreeProduct("Free Course");

console.log(freeProduct.price);

```

এখানে আমরা একসাথে ব্যবহার করেছি:

- class
- constructor
- instance property
- private field
- getter
- setter
- static property
- static factory method

43. Common Mistakes

Mistake 1 — Static method instance দিয়ে call করা

```
class User {  
  static create() {}  
}  
  
const user = new User();  
  
user.create();
```

❌ ভুল।

সঠিক:

```
User.create();
```

Mistake 2 — Private field বাইরে access করা

```
class User {  
  #password = "secret";  
}  
  
const user = new User();  
  
console.log(user.#password);
```

❌ Syntax error।

Mistake 3 — `_property` কে সত্যিকারের **private** ভাবা

```
this._password
```

এটা private নয়।

True private:


```
this.#password
```

Mistake 4 — `in` এবং `Object.hasOwn()` একই ভাবা

```
key in obj
```

own + inherited check করে।

```
Object.hasOwn(obj, key)
```

শুধু own property check করে।

Mistake 5 — `instanceof` দিয়ে সব **type check** করা

Primitive:

```
"hello" instanceof String
```

সাধারণত `false` হবে, কারণ primitive string এবং `String` wrapper object এক জিনিস নয়।

Primitive type-এর জন্য:

```
typeof value
```

ব্যবহার করো।

Array-এর জন্য:

```
Array.isArray(value)
```

44. Output Prediction

Example 1

```
class User {  
  static role = "user";  
  
  constructor(name) {  
    this.name = name;  
  }  
}  
  
const user = new User("Duke");  
  
console.log(User.role);  
console.log(user.role);
```

Output:

```
user  
undefined
```

Example 2

```
class User {}  
  
class Admin extends User {}  
  
const admin = new Admin();  
  
console.log(admin instanceof Admin);  
console.log(admin instanceof User);  
console.log(admin instanceof Object);
```

Output:

```
true  
true  
true
```

Example 3

```
const parent = {  
  role: "admin"  
};  
  
const user = Object.create(parent);  
  
user.name = "Duke";  
  
console.log("role" in user);  
console.log(Object.hasOwn(user, "role"));
```

Output:

```
true  
false
```

Example 4

```
class User {  
  #secret = 123;  
  
  getSecret() {  
    return this.#secret;  
  }  
}  
  
const user = new User();  
  
console.log(user.getSecret());
```

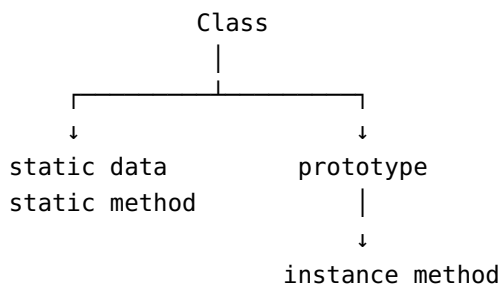
Output:

```
123
```

45. Quick Comparison

Feature	Purpose
<code>static</code>	Class-level behavior/data
<code>#field</code>	Truly private field
<code>#method()</code>	Private method
<code>instanceof</code>	Prototype-chain instance check
<code>Object.hasOwn()</code>	Own property check
<code>hasOwnProperty()</code>	Older own property check
<code>in</code>	Own + inherited property check
Mixins	Reusable behavior composition

46. `static` vs Instance



Example:

```
class User {  
  static type = "USER";  
  
  greet() {  
    return "Hello";  
  }  
}
```

Use:

```
User.type;
```

and:

```
const user = new User();  
  
user.greet();
```

47. Private State Mental Model

```
Object  
|  
├─ public properties  
|  
├─ public methods  
|  
└─ private fields (#)  
    ↓  
    class-এর ভিতরে access
```

Private state encapsulation তৈরি করতে সাহায্য করে।

48. Prototype Chain + instanceof Mental Model

ধরো:

```
class Animal {}  
  
class Dog extends Animal {}  
  
const dog = new Dog();
```

Chain:

```
dog  
↓  
Dog.prototype
```

```
↓  
Animal.prototype  
↓  
Object.prototype  
↓  
null
```

তাই:

```
dog instanceof Dog
```

→ true

```
dog instanceof Animal
```

→ true

```
dog instanceof Object
```

→ true

49. Property Ownership Mental Model

```
user  
├── name      ← own  
└── prototype  
    └── role ← inherited
```

তখন:

```
Object.hasOwn(user, "name")
```

→ true

```
Object.hasOwn(user, "role")
```

→ false

কিন্তু:

```
"role" in user
```

→ true

50. Interview Questions

Static

1. `static` method কী?
2. Static method instance দিয়ে কেন call করা যায় না?
3. Static property এবং instance property-এর difference কী?
4. Static factory method কী?
5. Static methods কি inheritance পেতে পারে?

Private Fields

1. JavaScript-এ private field কীভাবে তৈরি করা হয়?
2. `_property` এবং `#property` -এর difference কী?
3. Private field কি `Object.keys()` -এ পাওয়া যায়?
4. Private methods কী?
5. Encapsulation কী এবং private fields কীভাবে সাহায্য করে?

instanceof

1. `instanceof` কী?
2. `instanceof` কীভাবে prototype chain ব্যবহার করে?
3. `instanceof` এবং `typeof` -এর difference কী?
4. `dog instanceof Animal` কেন `true` হতে পারে?
5. Array check করার জন্য `instanceof Array` বনাম `Array.isArray()` কোনটা prefer করবে?

Property Ownership

1. Own property কী?
2. Inherited property কী?
3. `Object.hasOwn()` কী?
4. `hasOwnProperty()` এবং `Object.hasOwn()` -এর difference কী?
5. `in` operator এবং `Object.hasOwn()` -এর difference কী?

Mixins

1. Mixin কী?
 2. Mixins কেন ব্যবহার করা হয়?
 3. Inheritance এবং composition-এর difference কী?
 4. Mixins-এর potential সমস্যা কী?
 5. কখন inheritance-এর পরিবর্তে mixin/composition ব্যবহার করবে?
-

51. Practice Tasks

Task 1 — Static Counter

একটি `User` class তৈরি করো।

প্রতিবার নতুন user তৈরি হলে static counter increment হবে।

Expected:

```
const user1 = new User("Duke");
const user2 = new User("Rahim");

console.log(User.getCount());
```

Output:

2

Task 2 — Private Balance

`BankAccount` class তৈরি করো।

Private:

`#balance`

Methods:


```
deposit()  
withdraw()  
getBalance()
```

Rules:

- negative deposit allowed নয়
- balance-এর চেয়ে বেশি withdraw allowed নয়
- balance বাইরে থেকে direct access করা যাবে না

Task 3 — instanceof

Classes তৈরি করো:

```
Animal  
Dog  
Cat
```

Dog এবং Cat → Animal extend করবে।

তারপর predict করো:

```
dog instanceof Dog  
dog instanceof Animal  
dog instanceof Cat  
dog instanceof Object
```

Task 4 — Own Property Checker

Function:

```
hasOwn(obj, key)
```

তৈরি করো।

এটা Object.hasOwn() ব্যবহার করবে।

Task 5 — Mixins

দুটি behavior তৈরি করো:

```
canLogin  
canLogout
```

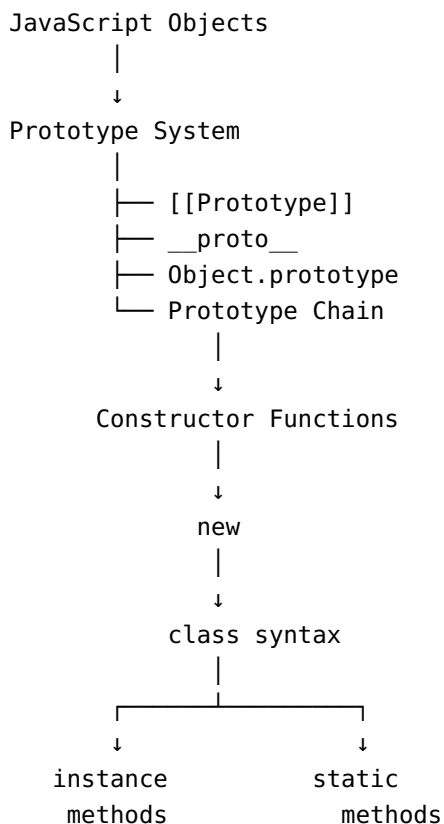
তারপর `User` class-এ দুইটি behavior add করো।

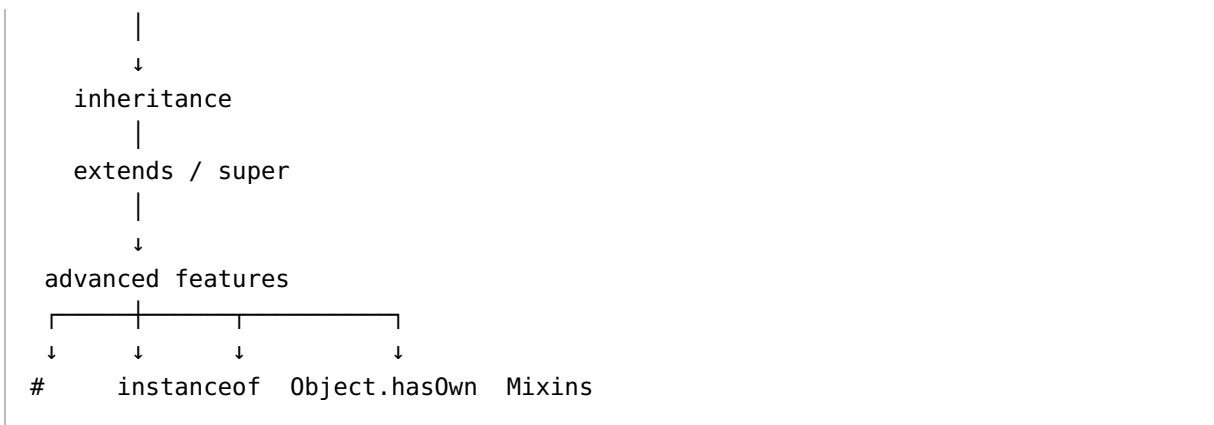
Expected:

```
user.login();  
user.logout();
```

52. Final Mental Model

এই পুরো **Prototype & Classes** section-কে এভাবে মনে রাখো:





53. Golden Rules

Rule 1

JavaScript fundamentally prototype-based ।

Rule 2

`class` prototype system-এর ওপর cleaner syntax দেয় ।

Rule 3

Instance method সাধারণত `Class.prototype` -এ থাকে ।

Rule 4

`static` member class-এর জন্য, instance-এর জন্য নয় ।

Rule 5

`#field` সত্যিকারের private ।

Rule 6

`instanceof` prototype chain check করে ।

Rule 7

`Object.hasOwn(obj, key)` শুধু own property check করে ।

Rule 8

`key in obj` own এবং inherited দুই ধরনের property-ই detect করতে পারে।

Rule 9

Mixins reusable behavior composition-এর জন্য useful।

Rule 10

Simple problem হলে simple solution ব্যবহার করো; unnecessary inheritance/mixins দিয়ে architecture complex করো না।

54. Complete Section Checklist

এই section শেষ করার পর তুমি explain করতে পারবে:

- [] `static` method
 - [] `static` property
 - [] static factory method
 - [] static inheritance
 - [] private fields `#`
 - [] private methods
 - [] private static fields
 - [] encapsulation
 - [] `instanceof`
 - [] prototype chain এবং `instanceof`
 - [] `typeof` বনাম `instanceof`
 - [] own property
 - [] inherited property
 - [] `hasOwnProperty()`
 - [] `Object.hasOwnProperty()`
 - [] `in` operator
 - [] `Object.hasOwnProperty()` বনাম `in`
 - [] Mixins
 - [] Mixin factory pattern
 - [] composition বনাম inheritance
 - [] advanced class design
-

Final One-Line Summary

`static` class-level behavior দেয়, `# true private state` তৈরি করে, `instanceof` prototype chain দিয়ে instance check করে, `Object.hasOwnProperty()` own property check করে, আর Mixins reusable behavior compose করতে সাহায্য করে।